Unitify Condo Integration

Connects **Unitify Condo** (the property-management ERP at `eu.unitify.com`) with the Newo AI agent so residents can open maintenance tickets, check ticket status, hear updates from the property team, update or close their requests, and submit / read utility meter readings — entirely by voice or chat, no front-desk required.

What the AI can do

- **Identify the resident by phone.** On every conversation the agent matches the caller's phone against the Unitify resident registry and silently loads the resident's profile (name, building, apartment, contact id) so the rest of the dialog is personalised. No "what's your apartment number?" questions when the registry already has the answer.
- **Open a maintenance ticket** when a resident reports a problem (leak, broken appliance, doorbell, lighting, common-area issue). The agent collects the issue description, picks the right classifier from the building's catalog, marks emergency vs routine, and writes the ticket to Unitify Condo so dispatch sees it immediately.
- **Read the resident's recent tickets back to them**, including the building team's latest update on each ticket — so the resident can ask *"когда придёт мастер?"* and hear the planned visit window without waiting on hold.
- **Update an existing ticket** — append new context the resident remembers ("the leak got worse"), escalate urgency, or correct details. The agent picks the right ticket from the resident's recent list by topical match (e.g. *"обнови ту про кран"* → the faucet ticket).
- **Close or cancel a ticket** when the resident no longer needs it ("сосед уже починил", "called by mistake"). Distinguishes *close* (resolved) from *cancel* (withdrawn), refuses to re-cancel a ticket that is already in a terminal status.
- **Read meter values** the building has on file (cold water, hot water, electricity) when the resident asks *"какие у меня показания?"*.
- **Submit fresh meter readings** when the resident dictates them ("холодная 12345, горячая 8901, электричество 22334") — the agent extracts each value, matches it to the right meter for the unit, and writes them to Unitify in one round-trip.
- **Hand off to a human** when the caller's phone is not in the registry (transfers the call to a 24/7 dispatcher line or relays the request to a managed mailbox).

Not in scope

- **Payments and invoicing.** Unitify's Acquiring (payment processor) module is not provisioned on the launch tenant, so the integration intentionally ships with the *Create Payment Link*, *Check Payment Status*, and *Cancel Invoice* features turned off (read-only, locked).
- **Smart-access / guest passes.** The smart-building module is not deployed on the launch tenant.
- **Resident-facing announcements.** The announcements module is out of scope for the first ship.

These can be re-enabled in a future release once the corresponding Unitify modules are provisioned.

Features at a glance

Feature	Included
Identify resident by phone (silent lookup)	✅
Create maintenance ticket	✅

Pick the right ticket classifier from the building's catalog	✅
Mark emergency vs routine	✅
Read recent tickets back to the resident	✅
Read the latest building-team update per ticket ("когда мастер?")	✅
Update an existing ticket (append details / escalate urgency)	✅
Close / cancel a ticket (with disambiguation by topic)	✅
Auto-resume cancel when caller asked to close pre-identification	✅
Refuse to cancel a ticket that is already closed / cancelled (idempotency)	✅
Read latest meter readings	✅
Submit fresh meter readings (multi-value in one turn)	✅
Auto-recognise voice / chat channel and route to the right transport	✅
Auto-handoff to human on registry miss (call transfer / email)	✅
OAuth 2.0 authorization-code flow with auto token refresh on 401	✅
Take payments / create invoices	❌ (Acquiring module not provisioned)
Issue smart-access guest passes	❌ (Smart-access module not provisioned)
Publish resident-facing announcements	❌ (out of scope for first ship)
Reschedule a ticket directly	❌ (cancel + create)

Scenarios

The integration ships with seven editable scenarios on the canvas. Operators tweak them in Newo Builder; the agent picks up the changes on the next **Publish All**.

What gets added on Publish All

Intent (canvas label)	Scenario (canvas heading)
Identify Resident in Unitify	Scenario 1: Identify Resident in Unitify
Create Service Request in Unitify	Scenario 2: Create Service Request in Unitify
Check Service Request Status in Unitify	Scenario 3: Check Service Request Status in Unitify

Update Service Request in Unitify	Scenario 4: Update Service Request in Unitify
Cancel Service Request in Unitify	Scenario 5: Cancel Service Request in Unitify
Read Meter Values in Unitify	Scenario 11: Read Meter Values in Unitify
Submit Meter Readings in Unitify	Scenario 12: Submit Meter Readings in Unitify

> ***Operational note (please read before upgrading).***
>
> 1. *These intents and scenarios are a **reference implementation** — they were tuned against the launch tenant and live test runs.*
> 2. *They are **fully editable** in the canvas UI. You can rewrite scenario steps, change the agent's tone, add clarifying questions, etc.*
> 3. ***Future integration updates will NOT overwrite an intent or scenario you have edited.** To pull the newer shipped defaults, delete your edited copy from the canvas and run **Publish All** — the original is restored from the integration's library, and you re-apply your tweaks on top of the newer baseline.*

Scenario 1 — Identify Resident in Unitify

Always runs first. Until the resident is identified the agent does nothing else.

1. Read the **Identification panel**. If it already shows *Status: identified*, briefly read the profile back (name + building + apartment + "is that right?") and ask how you can help.
2. If the panel is empty or *Status: awaiting_phone*, ask the resident for the phone number registered in the building system, in **E.164** format (`+998…`). Ask **only** for the phone — never name, address, apartment, building, or email.
3. Once the resident gives the phone, on the very next reply say (in the resident's language): *"Recording your phone and checking the resident registry, one moment…"* / *"Записываю ваш номер и проверяю реестр жильцов, минуту…"*. The phrase MUST be present-tense, first-person — past-tense / passive wording will not trigger the lookup.
4. While *Status: lookup_in_progress*, give the resident a brief reassuring line and stay on this step.
5. When the panel flips to *Status: identified*, paraphrase the profile (name + building + apartment) back, ask "is that right?". On confirmation, pick the appropriate feature scenario based on the request.
6. On *Status: not_found_in_registry*, apologise and hand off to the human manager (transfer call on phone, email on chat).

Trigger phrases (do not edit):

- *"Recording your phone and checking the resident registry, one moment…"* (or *"Записываю ваш номер и проверяю реестр жильцов, минуту…"*).

Triggered by intent **"Identify Resident in Unitify"**.

Scenario 2 — Create Service Request in Unitify

Runs when a resident reports a maintenance / service issue.

1. **Identification gate.** If the Identification panel is anything other than *identified*, switch to Scenario 1 and stop.
2. Acknowledge the issue and collect clarifying details: location inside the unit, when it started, water / power involved.

3. Confirm property and unit (already on the persona — no need to re-ask).
4. Reconfirm name and phone.
5. Ask the **emergency** question: *"Is this something that needs handling today, or can it wait for the next business day?"*.
6. Read the request back (issue + property/unit + contact + urgency) and ask: *"Should I submit this maintenance request now?"*.
7. On confirmation say: *"I'm submitting your maintenance request now. Please give me a moment, and I'll get back to you shortly."* — the agent files the ticket in Unitify Condo, picks the right classifier from the building's catalog, and reads the new ticket number back to the resident.

Trigger phrases (do not edit):

- *"I'm submitting your maintenance request now. Please give me a moment, and I'll get back to you shortly."*

Triggered by intent **"Create Service Request in Unitify"**.

Scenario 3 — Check Service Request Status in Unitify

Runs when the resident asks for the status of their requests, or *"когда придёт мастер?"*.

1. **Identification gate.**
2. Say: *"Let me check on your maintenance requests. Please give me a moment."* — the agent fetches the resident's most recent tickets and the latest building-team comment on each.
3. Read back **every** ticket: number, status, the issue description, any urgency flag, **and the latest update from the building team if one exists** (e.g. "the master will visit between 12:00 and 14:00 tomorrow"). When the resident asks *"when's the master coming?"* the latest-update line is the answer.
4. Offer follow-up: would they like to update or close any of them?

Trigger phrases (do not edit):

- *"Let me check on your maintenance requests. Please give me a moment." | "Сейчас проверяю ваши заявки, минуту..."*.

Triggered by intent **"Check Service Request Status in Unitify"**.

Scenario 4 — Update Service Request in Unitify

Runs when the resident wants to add information to or escalate an existing ticket.

1. **Identification gate.**
2. The agent picks the right ticket — by ticket number if the resident named one (*"обнови 803"*), or by topical match against the ticket details (*"обнови ту про кран"* → the faucet ticket). When several tickets match a topic, the most-recent **non-terminal** ticket wins.
3. Read the existing ticket back: number, current status, current description.
4. Ask: *"What would you like me to add to this request?"*. Capture the resident's wording verbatim.
5. Reconfirm: *"I will add the following to ticket [number]: '[appended text]'. Should I send this update?"*.
6. On confirmation say: *"I'm updating your existing request with these details now. Please give me a moment."* — the agent writes the update to Unitify and confirms the new status to the resident.

Trigger phrases (do not edit):

- *"I'm updating your existing request with these details now. Please give me a moment." | "Я сейчас обновляю вашу заявку, минуту..."*.

Triggered by intent **"Update Service Request in Unitify"**.

Scenario 5 — Cancel Service Request in Unitify

Runs when the resident wants to close or cancel an existing ticket.

1. **Identification gate.** If the resident asked to cancel BEFORE identification, the agent first asks for the phone, runs the identification flow, and on the next turn re-confirms the cancellation request before firing.
2. The agent picks the target ticket (explicit number or topical match — same rules as Update). Most-recent non-terminal ticket wins on a tie.
3. Read the ticket back and ask: *"Are you sure you want to close this request?"*. If the resident wants to add info instead, switch to Scenario 4.
4. On confirmation say: *"Okay, I'll close that request for you now. Please give me a moment." | "Закрываю вашу заявку прямо сейчас, минуту..."*.
5. The agent confirms the new status. If the ticket was already closed earlier, the agent says so and skips the cancel without errors.

Trigger phrases (do not edit):

- *"Okay, I'll close that request for you now. Please give me a moment." | "Закрываю вашу заявку прямо сейчас, минуту..."*.

Triggered by intent **"Cancel Service Request in Unitify"**.

Scenario 11 — Read Meter Values in Unitify

Runs when the resident asks for their **stored** meter readings (*"какие у меня показания записаны?"*).

1. **Identification gate.**
2. Confirm property and unit if missing.
3. Say: *"Let me pull up your meter readings. Please give me a moment."*.
4. Read back per resource type: cold water, hot water, electricity, etc. — current value, unit, and date of last submission.

Trigger phrases (do not edit):

- *"Let me pull up your meter readings. Please give me a moment."*.

Triggered by intent **"Read Meter Values in Unitify"**.

Scenario 12 — Submit Meter Readings in Unitify

Runs when the resident dictates **fresh** meter values to be saved.

1. **Identification gate.**
2. Confirm property and unit if missing.
3. Capture the readings the resident dictates. Read them back verbatim per resource: *"cold water 12345, hot water 8901, electricity 22334 — is that right?"*. Do NOT paraphrase numbers.
4. Wait for explicit confirmation. If a value is wrong, update the recap and re-confirm.
5. On confirmation say: *"I'm submitting your meter readings now. Please give me a moment."* — the agent writes all the readings in one round-trip and confirms each saved value back to the resident.

Trigger phrases (do not edit):

- *"I'm submitting your meter readings now. Please give me a moment."*.

Triggered by intent **"Submit Meter Readings in Unitify"**.

Customizing the agent

For every scenario:

- ✅ Anything outside the trigger phrases is safe to reword (clarifying questions, tone, examples).
- ❌ Trigger phrases (in italics in each scenario above) are the literal phrases that fire the integration's actions. If you change them, the integration stops responding.
- ❌ Do not delete the intent on the canvas — without the intent, the scenario is never selected.
- ↩️ To restore the shipped scenario, delete your edited copy and run **Publish All** — the original is re-published from the library.

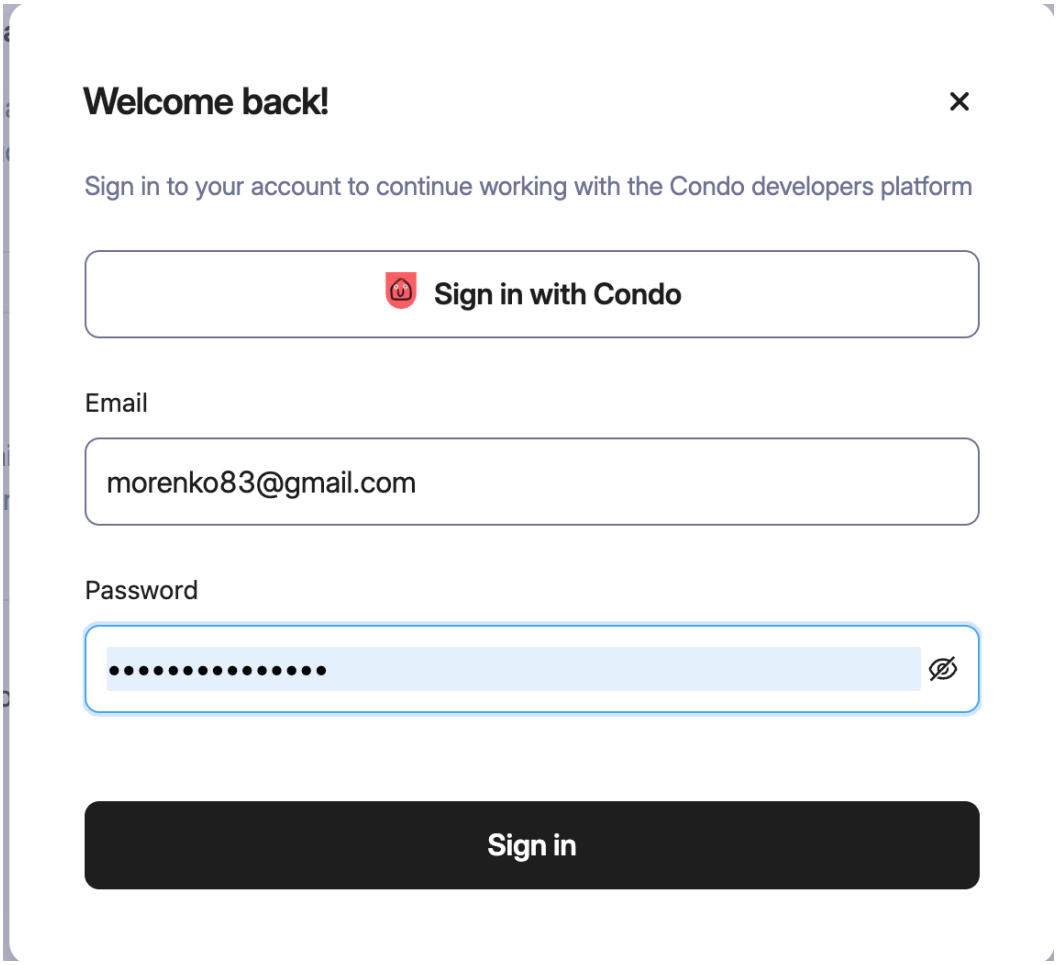
Setup

A step-by-step walkthrough is below. Two halves: first, prepare your Unitify side; second, configure Newo.

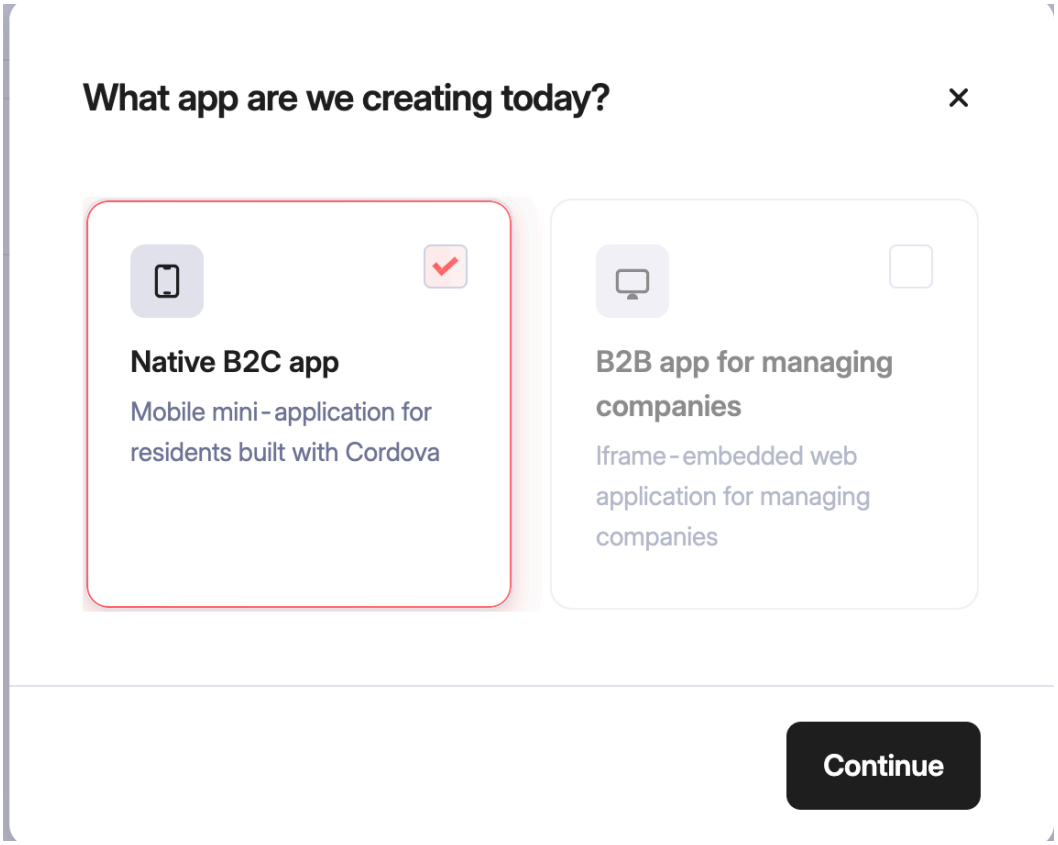
1. Register a developer application in Unitify

You need an OAuth 2.0 application registered on Unitify's developer portal. The application gives Newo a **Client ID + Client Secret** that the integration uses to obtain a Bearer token (auto-refreshed on every 401).

1. Open the **Unitify developer portal** and sign in with the building-manager account that has rights to create tickets, read meters, and (if applicable) publish announcements.

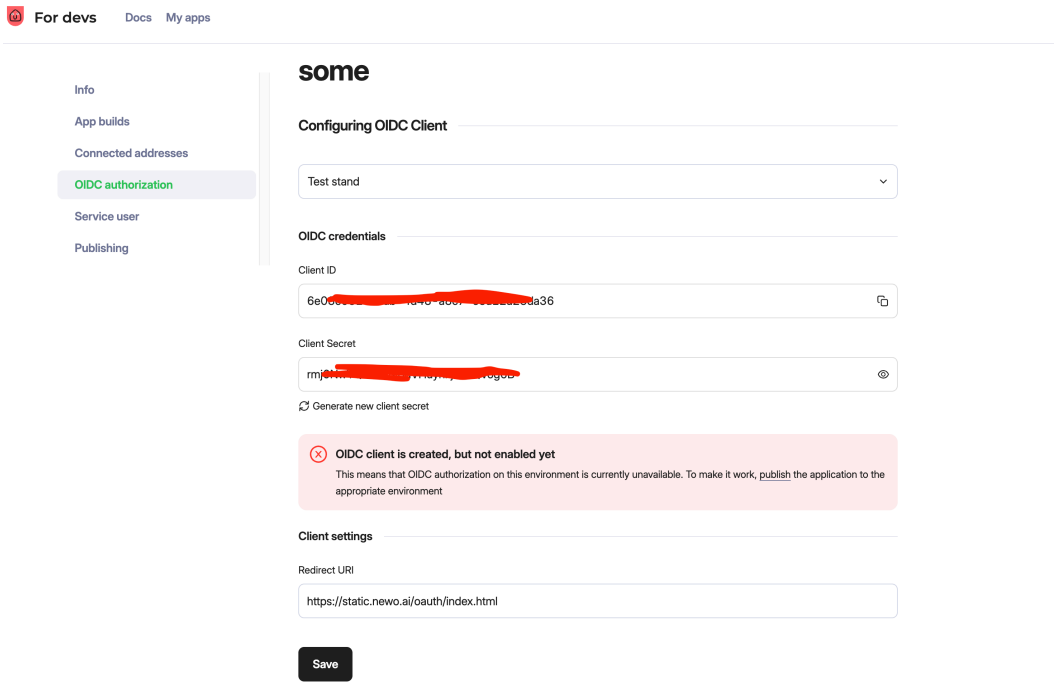


2. Open **My apps** on the left side menu, then click **Create mini-app**. In the dialog, select **Native B2C app** and click **Continue**.

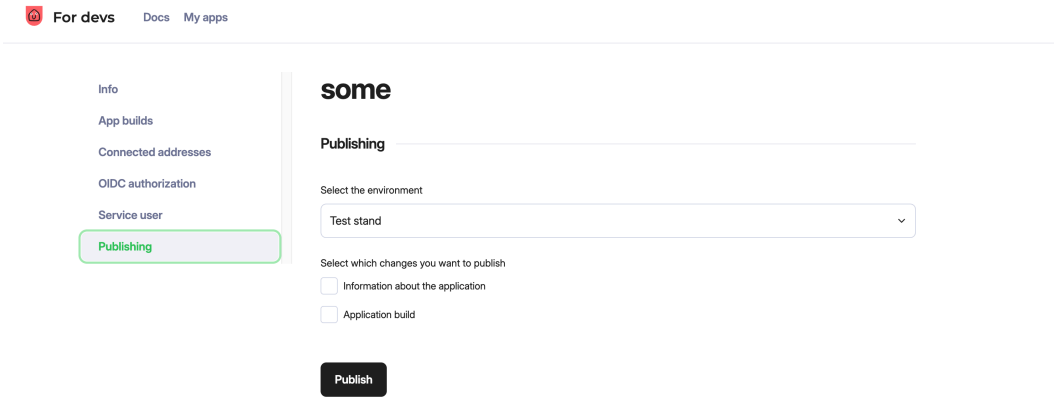


3. Fill in the new-app form:

   - **What shall we call the new app?** — anything readable, e.g. `Newo Voice Agent`. Residents never see this.
   - Choose **OIDC authorization** in the left side menu and click **Create**.
   - Fill **Redirect URI** — paste exactly: `https://static.newo.ai/oauth/index.html`. This is the Newo-hosted helper page that captures the OAuth code after consent. Click **Create**.
   - The portal now displays the **Client ID** and **Client Secret**. Copy both and store them safely, then click **Save**.



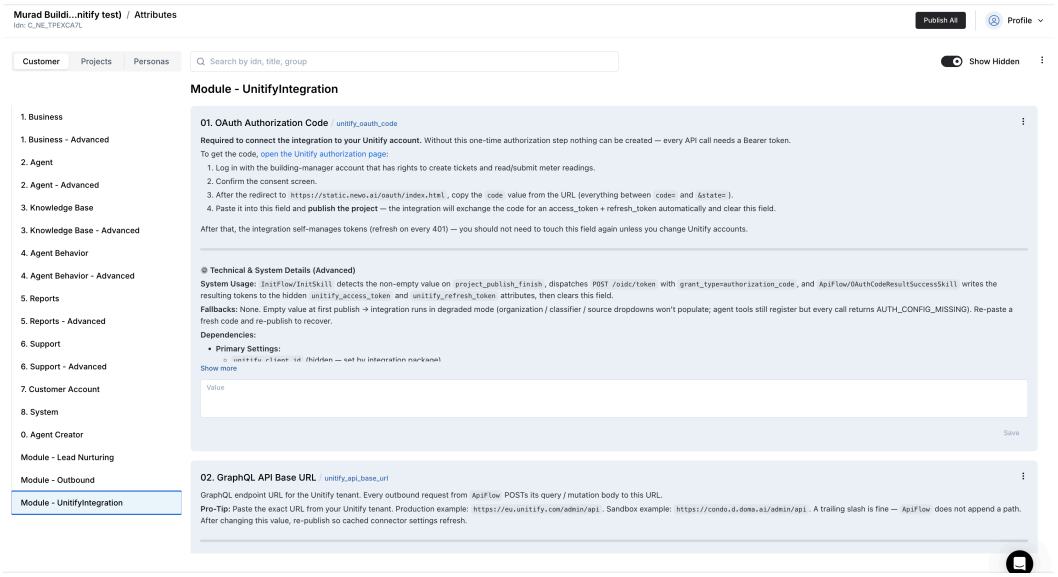
4. Choose **Publishing** on the left side menu and click **Publish** so the application becomes usable.



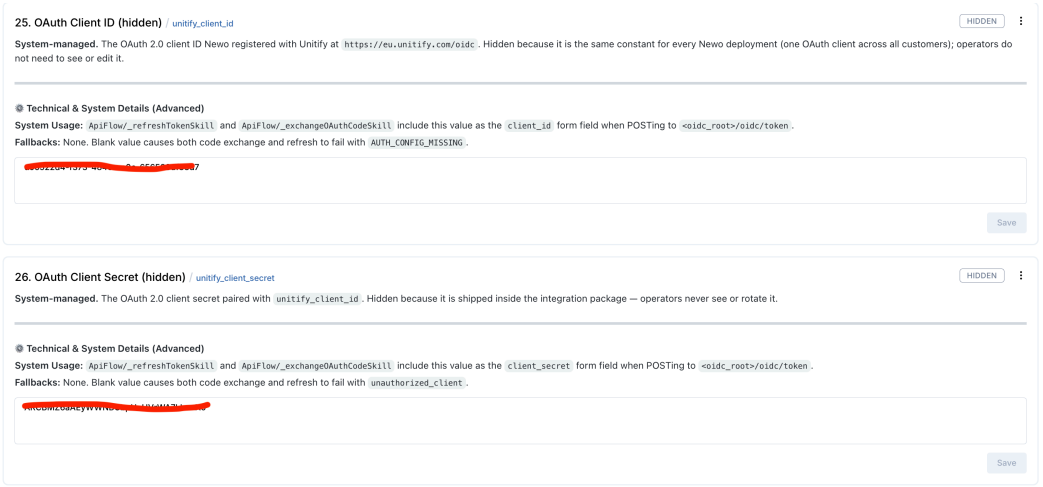
5. Note for the next step:
    - Paste the **Client ID** into Newo as **OAuth Client ID (hidden)**.
    - Paste the **Client Secret** into Newo as **OAuth Client Secret (hidden)**.
    - Confirm the **GraphQL endpoint URL** for your tenant. The default for the EU production cluster is `https://eu.unitify.com/admin/api`; sandbox / regional clusters differ.

2. Configure the integration in Newo

1. Open the project in **Newo Builder** and find the **Module - UnitifyIntegration** attribute group.

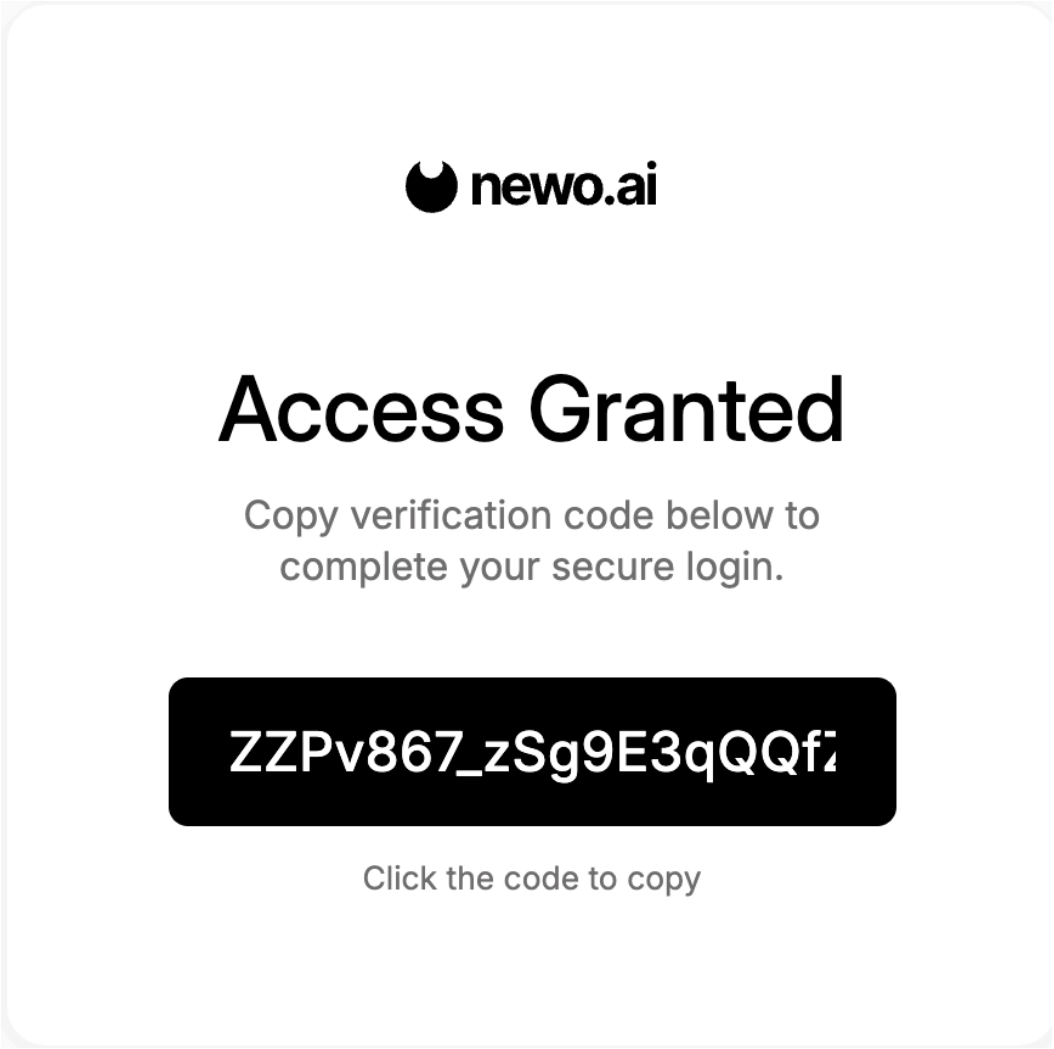


2. Paste the credentials you just copied:
    - **OAuth Client ID (hidden)** ← Client ID from Unitify.
    - **OAuth Client Secret (hidden)** ← Client Secret from Unitify.
    - **GraphQL API Base URL** ← Unitify GraphQL endpoint (default `https://eu.unitify.com/admin/api`; change only for sandbox / regional clusters).



3. Click **Publish All**. On this first publish the integration:
    - Provisions all the operator-facing settings.
    - Builds the **Unitify authorization URL** and inlines it into the description of the **OAuth Authorization Code** setting.
4. Open the **OAuth Authorization Code** setting. Click the link in its description to open the Unitify authorize page, sign in with the building-manager account, and confirm the consent screen. Unitify redirects you to `https://static.newo.ai/oauth/index.html?code=…&state=…`.

5. Click on code field return to newo.ai and paste it into the **OAuth Authorization Code** field in Newo and click **Save**. Click **Publish All** again.



   On this publish the integration exchanges the code for an access token and a refresh token, stores them in the hidden token attributes, and clears the **OAuth Authorization Code** field. After this you should never need to touch this field again — token rotation happens automatically on 401.

6. Click **Publish All** again.

7. The integration also auto-discovers the building's reference data on this publish:

   - **Organization (Tenant)** — populated as a dropdown of `id : name` values pulled from `allOrganizations`.
   - Hidden caches: ticket statuses, ticket sources, meter-reading sources, classifiers (so the agent can pick the right one without an extra round-trip on every ticket).

8. Pick your tenant from the **Organization (Tenant)** dropdown and click **Publish All** one more time. This is the last publish required for first install.

   The agent is now ready to take a real conversation. Place a test chat message ("the doorbell isn't working — open a request") to confirm.

3. Optional — review the feature toggles

Every feature can be turned on or off independently. Defaults are sensible, but operators commonly review:

Setting (UI label)	What it gates	Default
Auto-Setup Canvas Scenarios	Publishes the seven scenarios + their intents on first install. Disable only when you maintain a fully custom canvas.	on
Create Service Request Feature	Master on/off for opening tickets.	on
Check Request Status Feature	Master on/off for the status read-back (and the latest-update relay).	on
Cancel / Close Ticket Feature	Master on/off for ticket cancellation.	on
Update Service Request Feature	Master on/off for appending details to tickets.	on
Meter Readings Feature	Master on/off for reading stored meter values.	on
Submit Meter Readings Feature	Master on/off for accepting fresh meter values.	on

Disable a feature → its custom tool stops being offered to the agent and the corresponding scenario goes idle. Re-enable + **Publish All** to bring it back.

How to test that everything works

After the third publish, run these mini-checks in the Newo test chat (or by calling the agent's number).

1. **Identification.** Say: *"Hi, what apartment am I in?"* The agent asks for your phone in E.164 format. Reply with a phone you registered in Unitify Condo. Within a few seconds the agent reads back your name + building + apartment.
2. **Create a ticket.** Say: *"The doorbell isn't working, please open a request."* After the identity readback the agent asks clarifying questions, says *"I'm submitting your maintenance request now…"*, and reads back a brand-new ticket number. **Verify in Unitify Condo:** the ticket appears under your contact, with a sensible classifier.
3. **Check status + latest update.** In Unitify Condo, add a *resident*-type comment on a ticket (e.g. *"the master will visit at 12:00–14:00 tomorrow"*). Then in a new chat, after identification, ask *"когда придёт мастер?"*. The agent says *"Let me check…"* and then reads back the ticket list **with the latest update line per ticket**.
4. **Update a ticket.** Say: *"Add to ticket 808 that it's gotten worse — water on the floor."* After identification + readback the agent says *"I'm updating your existing request now…"* and confirms the new ticket details. **Verify in Unitify Condo:** the ticket's `details` field now contains the appended text.
5. **Cancel a ticket.** Say: *"Close the request about the doorbell — neighbour will fix it."* After identification + readback + the resident's confirmation the agent says *"Закрываю вашу заявку прямо сейчас, минуту…"* and confirms the closed status. **Verify in Unitify Condo:** the ticket moves to a Closed / Canceled status.
6. **Submit meter readings.** Say: *"I want to submit my meter readings — cold water 12345, hot water 8901, electricity 22334."* The agent reads the values back, asks for confirmation, says *"I'm submitting your meter readings now…"*, and confirms each saved value. **Verify in Unitify Condo:** new MeterReading records appear under your unit with the dictated values.
7. **Read meter values.** In a fresh session, say *"What are my latest meter readings?"*. After identification the agent reads back per-resource current values + their date.

If a step fails, the most common fix is to repaste the OAuth Client ID + Secret + run the OAuth-code flow again, then **Publish All**.

Customisation cheat-sheet

What you can fine-tune without touching code:

- **Scenarios on the canvas.** Reword steps, add clarifying questions, change tone. Keep the trigger phrases verbatim (see each scenario block above).
- **Feature toggles.** Disable / enable any of the seven features per the table above.
- **Manager handoff targets.** Set **Manager Phone Number (handoff target)** for voice transfers and **Manager Email (handoff target)** for chat handoffs — used when the caller's phone is not in the registry.
- **Default property fallback. Default Property ID (optional)** — only set this for single-building tenants where address ambiguity is impossible. Leaving it blank is the safer default.
- **Sender fingerprint. Sender Fingerprint** — what shows up on every Unitify mutation in the audit log. Default `newo_unitify_integration` is fine; override only if your Unitify ops team asks for a tenant-specific tag.
- **Ticket status overrides** (advanced). The integration auto-discovers the tenant's `Open` / `Closed` / `Cancelled` status records via `allTicketStatuses`. If your tenant has multiple records of the same kind (e.g. `Closed — Resolved` vs `Closed — Withdrawn`) and you want the agent to use a specific one, paste its UUID into the corresponding **Ticket Status Override** field. Otherwise leave blank and let auto-discovery pick.

Frequently asked questions

Q. Does the agent ever charge the resident? No. Payments are intentionally not in scope on the launch tenant — the Acquiring module is not provisioned. The agent only opens / updates / cancels tickets and reads or submits meter values.

Q. Two residents share one phone number — what happens? Unitify resolves a single contact per phone. If two residents share, the agent uses whichever profile the registry returns (typically the primary contact). The agent reads back the profile so the caller can correct course (or hand the call to a human) before any ticket is created.

Q. The resident says *"когда придёт мастер?"* but the team has not commented yet — what does the agent say? The agent reads the ticket back and explicitly says *"no resident updates yet"* on that ticket. Operators can drop a one-line comment in Unitify Condo any time; the next status check picks it up automatically.

Q. What if the resident is not in the registry? The agent apologises and routes to a human — call transfer (`transfer_call_tool`) on phone, email (`send_email_tool`) on chat. Targets come from the **Manager Phone Number / Manager Email** settings.

Q. The resident wants to update an old, closed ticket — what happens? For Update / Cancel, the integration prefers the **most-recent non-terminal** match by topic. So *"обнови про кран"* lands on the active faucet ticket, not the closed one from last month. If the resident genuinely means a specific older ticket, they can name it by number ("обнови 803").

Q. The resident dictates meter readings but a meter is not registered for the unit — what happens? The agent tries to write what it can and reports back per-resource: which readings were saved, which were rejected, and why. It then re-asks for the rejected values or routes the resident to a human.

Q. What happens if the OAuth tokens expire or are revoked? The integration auto-refreshes on every 401 using the refresh token — no operator action needed. Only if both tokens are revoked (e.g. the building-manager account was disabled in Unitify) does the operator have to repaste a fresh **OAuth Authorization Code**.

Q. We added a new ticket classifier in Unitify Condo. When does the agent see it? On the next **Publish All** — the integration re-fetches the classifier index on every publish.

Q. Can the agent answer policy questions ("what's the after-hours emergency line?")? Those come from the **Business Context** on the canvas, not from Unitify. Edit that block to surface tenant-specific policy.

Q. Can I run two Unitify tenants from one Newo project? One tenant per integration instance. To serve two tenants, deploy two Newo projects.

Limitations

- **Reschedule** is not a separate operation — the resident cancels the open ticket and the team opens a new one when needed.
- **Payments / invoices**, **smart-access guest passes**, **resident announcements** — intentionally not shipped on the launch tenant; can be re-enabled when Unitify provisions the matching modules.
- **Single tenant per integration instance.**
- **Phone-only identification.** The agent does not fall back to name / address / email lookup — Unitify uses phone as the primary key for residents. If a resident's phone is wrong in Unitify, fix it in Unitify Condo first.

All settings reference

Every attribute the integration adds to the **Module - UnitifyIntegration** group in Newo Builder. Operators typically only touch the rows marked ✅; ⚠️ rows are advanced (hidden by default, edit only when troubleshooting); 🔒 rows are managed automatically and should not be edited by hand.

Name	Description	Required	
`unitify_oauth_code`	One-time OAuth authorization code returned by Unitify after consent. Cleared automatically after the first successful exchange.	Yes (first publish)	*(e*
`unitify_api_base_url`	GraphQL endpoint URL for the Unitify tenant (e.g. `https://eu.unitify.com/admin/api`).	Yes	`ht`
`unitify_organization_id`	Tenant (organization) selected for every Unitify mutation. Auto-populated as a dropdown after the first successful auth.	Yes	*(e*
`unitify_sender_fingerprint`	Audit-log identifier written into every Unitify mutation. Override only if your Unitify ops team asks for a tenant-specific tag.	No	`ne`
`unitify_default_property_id`	Optional fallback Property ID used only when the resident does not disclose an address and `persona_attributes_unitify_property_id` is empty. Leave blank for multi-building tenants.	No	*(e*
`unitify_manager_phone_number`	Phone number to transfer the call to when a caller is not in the Unitify registry. Use full E.164 format.	No (recommended)	*(e*
`unitify_manager_email`	Email address to forward chat conversations to when a chat user is not in the registry. Should be a managed inbox.	No (recommended)	*(e*
`unitify_setup_scenarios`	Publishes the seven Unitify scenarios + intents on every publish. Disable only if you maintain a fully custom canvas.	No	or
`unitify_enable_create_ticket`	Master on/off for opening maintenance tickets.	No	or
`unitify_enable_check_ticket`	Master on/off for reading the resident's tickets and the latest building-team updates back to them.	No	or
`unitify_enable_cancel_ticket`	Master on/off for closing / cancelling tickets.	No	or

`unitify_enable_update_ticket`	Master on/off for appending details to existing tickets / escalating urgency.	No	or
`unitify_enable_meters`	Master on/off for reading stored meter values.	No	or
`unitify_enable_submit_meters`	Master on/off for accepting fresh meter readings dictated by the resident.	No	or
`unitify_ticket_status_open_id`	Optional override for the `TicketStatus` UUID used when opening a new ticket. Blank → auto-discovery picks the first record of `type = open`. Set only when your tenant has multiple `open`-typed statuses and you need a specific one.	No	*(e*
`unitify_ticket_status_closed_id`	Optional override for the `TicketStatus` UUID used when the resident says *close*. Blank → auto-discovery.	No	*(e*
`unitify_ticket_status_canceled_id`	Optional override for the `TicketStatus` UUID used when the resident says *cancel*. Blank → auto-discovery.	No	*(e*
`unitify_ticket_classifier_index`	Hidden cache of the tenant's ticket classifiers. Auto-populated on every publish; do not edit by hand.	No	`[]`
`unitify_ticket_sources_index`	Hidden cache of the tenant's ticket sources (call / messenger / e-mail / etc). Auto-populated on every publish.	No	`[]`
`unitify_meter_reading_sources_index`	Hidden cache of the tenant's meter-reading sources. Auto-populated on every publish.	No	`[]`
`unitify_client_id`	OAuth 2.0 Client ID from the Unitify developer portal.	Yes	*(e*
`unitify_client_secret`	OAuth 2.0 Client Secret from the Unitify developer portal.	Yes	*(e*
`unitify_redirect_uri`	OAuth redirect URI registered with the Unitify app. Newo-hosted helper page; do not change.	Yes	`ht`
`unitify_access_token`	Bearer token used for every Unitify GraphQL request. Auto-managed (refreshed on every 401).	Yes	*(e*
`unitify_refresh_token`	OAuth refresh token. Auto-rotated by the integration.	Yes	*(e*

Common errors and recovery

**"Publish All" fails with an authentication error

Likely cause. The OAuth Client ID, Client Secret, or one-time code is wrong, or the building-manager account does not have permission for the tenant.

Recover:

1. In Newo Builder, open **OAuth Client ID (hidden)** and **OAuth Client Secret (hidden)**. Re-copy them from the Unitify developer portal and paste again — make sure no leading / trailing whitespace.
2. Re-do the consent step on the Unitify portal and copy a fresh `code=…` value from the redirect URL.
3. Paste the code into **OAuth Authorization Code** and click **Publish All**.

Verify. The **OAuth Authorization Code** field is automatically cleared after a successful exchange, and the **Organization (Tenant)** dropdown is populated.

Organization (Tenant) dropdown is empty after Publish All

Likely cause. Tokens are missing or the OAuth code was already consumed.

Recover: repeat the OAuth-code paste step in *Setup → 2. Configure in Newo*, then **Publish All** again.

Agent stops opening tickets after I edited a scenario

Likely cause. The trigger phrase was reworded.

Recover: open the affected scenario on the canvas and restore the trigger phrase exactly as quoted in the *Scenarios* section above. Click **Publish All**. To roll back to the shipped baseline, delete the scenario from the canvas and **Publish All** — the original is restored from the integration's library.

Resident is not in the Unitify registry on every call

Likely cause. The phone format does not match what Unitify stores. Unitify stores `clientPhone` with the leading `+`; if a previous integration wrote phones without the `+`, the lookup misses.

Recover: in Unitify Condo, normalise the resident's phone to E.164 with a leading `+`, and ensure the contact record's phone matches what the carrier passes for that resident.

Agent says *"no resident updates yet"* after the team commented in Unitify

Likely cause. The comment was added with `type = organization` (internal). The agent only relays comments with `type = resident`.

Recover: in Unitify Condo, switch the comment type from *organization* to *resident* (or post a new resident-type comment) so it is visible to the agent.

Changelog

v1.3.0 — Maintenance-tickets MVP + meter readings + comments relay

- **Phone-based silent identification** via Unitify's `allContacts`. ExistingClientFlow pulls profile + property + unit on every conversation start; subscribes to NAF's `define_user_phone_number` event to read the carrier-provided phone (no LLM-mangled digits).

- **Create / Check / Update / Cancel ticket** end-to-end with auto-fetch + topical disambiguation. Update and Cancel pick the right ticket by ticket number when named, or by topical match against `details`; non-terminal status preferred on a tie.
- **Latest building-team comment relay.** Check Status fetches `allTicketComments(type: resident)` for every returned ticket and inlines the latest comment per ticket so the agent can answer *"when's the master coming?"* without an extra hop.
- **Meter readings** — read and submit (single-shot multi-resource).
- **OAuth 2.0 authorization-code flow** with refresh-token rotation on 401.
- **Auto-resume cancel after identification** — resident asking to cancel pre-identification triggers a phone-ask, identification, then a re-confirm step before the cancel actually runs. Avoids the "ticket closed before identity confirmed" UX bug.
- **NAF default scenarios scrubbed from canvas.** `[T] Client Support`, `[T] Complaints`, `[T] Contractor Support`, `[T] Other Services Request`, `[T] Emergency` (auto-published by NAF when `project_business_industry = home_services`) are deleted on every publish so they do not compete with the Unitify scenarios.
- Payments / invoices / announcements / smart-access — feature flows deliberately not shipped (matching modules not provisioned on the launch tenant).